

تمرین‌های کلاسی هفته سوم

موضوع کلی: ساخت یک موتور پیشنهاد مقصد سفر (Travel Match Recommender)

راهنمای نشانه‌ها:

Core و هسته اصلی

Stretch (کمی چالش‌برانگیزتر اما در محدودهٔ مطالب)

Search-required (نیاز به جستجو دارد – راهنمایی داده شده)

Bonus (اختیاری، الزامی نیست)

بخش 1 – گرم کردن

1. با داشتن

```
scores = [72, 85, 90, 66, 78],
```

آن را به یک آرایهٔ NumPy تبدیل کنید و مقادیر ndim, shape, size, dtype را چاپ کنید.

2. با داشتن:

```
grades = np.array([
    [18, 15, 12, 20], # Student 1
    [14, 17, 19, 16], # Student 2
    [20, 20, 18, 15] # Student 3
])
```

موارد زیر را استخراج کنید:

- کل سطر 2 Student
- ستون آخر (فرض کنید مربوط به «Final Exam» است)
- sub-matrix شامل 2 دانشجوی اول و 2 درس اول

3. با داشتن

```
a = np.array([3, 6, 9])
```

```
b = np.array([1, 2, 3]),
```

جمع، تفاضل و ضرب elementwise را حساب کنید؛ سپس a را در 0.5 ضرب کنید.

4. یک ماتریس 3×3 به نام X و یک بردار

```
v = np.array([100, 0, -5])
```

دارید. مقدار $v + X$ را محاسبه کنید و در یک جمله توضیح دهید NumPy چه کاری روی v انجام داد تا shape با هم سازگار شوند.

5. دو نقطه در فضای سه‌بعدی:

```
point_A = np.array([2, 3, 5])
```

```
point_B = np.array([7, 1, 9])
```

فاصله Euclidean را با `np.linalg.norm` محاسبه کنید. نتیجه به 5 نزدیک‌تر است یا به 10؟

6. ماتریس زیر امتیاز 4 دانش‌آموز را در 3 درس نشان می‌دهد:

```
M = np.array([
    [80, 70, 90],
    [60, 85, 75],
    [95, 60, 80],
    [70, 70, 70]
])
```

ترانپوز (transpose) این ماتریس را بگیرید و shape جدید را چاپ کنید. سپس M را با reshape یک بار به شکل (2, 6) و یک بار به شکل (6, 2) دریاورید. کدام یک از این دو reshape واقعاً معنای معقولی برای داده دارد و چرا؟

7. فرض کنید هر یک از 3 درس بالا یک ضریب متفاوت دارد:

```
weights = np.array([0.5, 0.3, 0.2])
```

با استفاده از np.dot، معدل وزن‌دار هر دانش‌آموز را در یک خط محاسبه کنید. سپس همان نتیجه را یک بار دیگر با ضرب elementwise و np.sum روی هر سطر به دست بیاورید و مطمئن شوید دو جواب یکسان‌اند.

بخش 2 – Travel Match Engine

گروه شما برای یک استارت‌آپ travel کار می‌کند. قرار است یک engine ساده بسازید که بر اساس ترجیحات سفر، یک شهر پیشنهاد بدهد.

Data setup – این فایل را با نام cities.csv ذخیره کنید:

```
city,avg_temp,humidity,popularity,cost_index
Kish,32,55,9,7
Shiraz,28,35,8,5
Tabriz,18,50,6,4
Yazd,30,20,5,3
Rasht,22,75,7,6
Isfahan,25,40,9,5
Mashhad,20,45,8,4
Ardabil,17,N/A,4,3
```

توجه: سطر آخر یک مقدار خراب دارد (N/A برای humidity) – دقیقاً مشابه چیزی که هفته قبل در file handling با آن کار کردید.

2A. بارگذاری و ساخت داده

1. فایل cities.csv را با open() و csv.reader (یا csv.DictReader) بخوانید و یک list of dictionaries بسازید.

2. تبدیل "N/A" به عدد برای سطر Ardabil خطا می‌دهد. این تبدیل را داخل try/except بگذارید تا به جای crash کردن برنامه، آن سطر skip شود و یک پیام warning چاپ شود.

3. بعد از پاک‌سازی، دو چیز بسازید: city_names (لیست ساده‌ای از نام شهرها) و data (آرایه NumPy شامل فقط ستون‌های عددی).

4. data.shape و data.dtype را چاپ کنید – باید 7 شهر معتبر و 4 feature داشته باشید.

2B. کاوش و مقیاس‌بندی داده

1. با استفاده از axis مناسب، mean، max، min و standard deviation هر یک از 4 feature را حساب کنید.

2. روی data، مقیاس‌بندی Min-Max اعمال کنید تا scaled_data به دست بیاید و همه مقادیر بین 0 و 1 قرار بگیرند.

3. به صورت گروهی بحث کنید: اگر قبل از محاسبه فاصله‌ها این scaling را انجام ندهید، چرا مقایسه فاصله خام بین شهرها می‌تواند گمراه‌کننده باشد؟ جواب را در یک کامنت یک‌خطی در نوت‌بوک بنویسید.

2C. اندازه‌گیری شباهت بین شهرها

1. سفر ایده‌آل خودتان را به عنوان گروه تعریف کنید، مثلاً:

```
ideal = np.array([26, 40, 8, 4])
```

آن را با همان min/max‌هایی که قبلاً محاسبه کرده‌اید scale کنید.

2. با broadcasting و بدون استفاده از for loop، فاصله Euclidean بین scaled_ideal و همه سطرهاى scaled_data را محاسبه کنید.

3. np.min() فقط کمترین فاصله را برمی‌گرداند اما مشخص نمی‌کند مربوط به کدام شهر است. تابعی در NumPy پیدا کنید که index کوچک‌ترین مقدار را برمی‌گرداند (راهنمایی: با np.arg... شروع می‌شود) و نام شهر پیشنهادی را چاپ کنید.

4. همچنین cosine similarity را برای همه شهرها حساب کنید و مشابه‌ترین شهر را پیدا کنید. آیا نتیجه با مرحله قبل یکی است؟ اگر نه، دلیلش را بررسی کنید.

2D. توصیه‌گر وزن‌دار

همه افراد به همه featureها اهمیت یکسانی نمی‌دهند.

1. یک آرایه از وزن‌ها تعریف کنید:

```
weights = np.array([0.1, 0.2, 0.3, 0.4])
```

(مجموع وزن‌ها باید 1 باشد).

2. با broadcasting، قبل از محاسبه فاصله، این وزن‌ها را روی scaled_data و scaled_ideal اعمال کنید.

3. تابع زیر را بنویسید:

```
def recommend(ideal, weights, data, city_names):
```

```
    نام تک شهر با بهترین تطابق را برمی‌گرداند #
```

4. تابع را طوری تغییر دهید که به جای یک شهر، 3 شهر برتر را برگرداند. برای این کار باید ترتیب مرتب‌شده فاصله‌ها را به دست بیاورید.

2E. بررسی vectorization

شما عمداً از for loop دوری کرده‌اید. حالا نشان دهید این کار چقدر روی سرعت تأثیر می‌گذارد.

```
def slow_weighted_distance(ideal, weights, data):
```

```
    results = []
```

```
    for row in data:
```

```
        total = 0
```

```
        for i in range(len(row)):
```

```
            total += weights[i] * (row[i] - ideal[i]) ** 2
```

```
        results.append(total ** 0.5)
```

```
    return results
```

1. با %%timeit این تابع را با نسخه vectorized خودتان مقایسه کنید.

2. اختلاف سرعت را گزارش کنید.

2F. ماتریس فاصله‌ها

تا اینجا فاصله هر شهر را فقط تا یک نقطه ایده‌آل حساب کرده‌اید. حالا فاصله همه شهرها را تا همه شهرهای دیگر حساب کنید.

1. با استفاده از np.newaxis، scaled_data را طوری broadcast کنید که یک ماتریس (7, 7) به دست بیاید که خانه [i, j] آن فاصله شهر i تا شهر j باشد – باز هم بدون for loop.

2. قطر اصلی این ماتریس باید همه صفر باشد؛ در یک کامنت توضیح دهید چرا.

3. برای هر شهر، نزدیک‌ترین «همسایه» (شهری غیر از خودش) را با np.argsort یا ترکیبی از np.argmax پیدا کنید.

بخش 3 – جمع‌بندی

یک cell markdown نهایی اضافه کنید و خلاصه‌ای بنویسید که شامل این دو مورد باشد:

- engine گروه شما چه شهری را پیشنهاد کرد؟
- چه چیزی شما را شگفت‌زده کرد؟

بخش 4 - تمرینات سبک تر

1. موجودی انبار فروشگاه (ماسک‌گذاری بولین / Boolean Masking)

```
prices = np.array([2.5, 12.0, 0.8, 45.0, 6.2, 3.1])
stock = np.array([120, 5, 300, 2, 40, 0])
```

با استفاده از یک شرط بولین واحد (بدون استفاده از حلقه)، مشخص کنید کدام کالاها هم «ارزان» هستند (قیمت کمتر از ۵) و هم «موجودی انبار آن‌ها کم» است (کمتر از ۱۰ عدد باقی مانده). در نهایت، قیمت این کالاها را چاپ کنید.

2. ایستگاه هواشناسی (مفهوم Axis + برادکستینگ)

```
temps = np.array([
    [22, 24, 19, 21], # دوشنبه، ۴ بار اندازه‌گیری در طول روز
    [25, 27, 23, 24], # سه‌شنبه
    [18, 20, 17, 19]  # چهارشنبه
```

الف) میانگین دما را برای هر روز و همچنین میانگین دما را برای هر بازه زمانی (ستون‌ها) در طول این سه روز محاسبه کنید. (راهنمایی: به این فکر کنید که برای هر کدام باید از کدام محور یا `axis` استفاده کنید).

ب) با استفاده از ویژگی برادکستینگ (Broadcasting)، در یک خط کد، میانگین هر روز را از اندازه‌گیری‌های همان روز کم کنید تا مشخص شود هر داده چقدر با میانگین روزانه خود فاصله دارد.

3. قطعه‌ای از یک تصویر خاکستری (ایندکس‌گذاری / اسلایسینگ)

```
image = np.array([
    [10, 20, 30, 40, 50],
    [15, 25, 35, 45, 55],
    [20, 30, 40, 50, 60],
    [25, 35, 45, 55, 65],
    [30, 40, 50, 60, 70]
])
```

یک محدوده 3×3 را درست از مرکز این تصویر استخراج کنید. سپس یک نسخه «روشن‌تر» از کل تصویر ایجاد کنید که در آن مقدار هر پیکسل ۲۰ واحد افزایش یافته باشد، اما سقف آن روی ۲۵۵ محدود شود (راهنمایی: از `np.clip` یا `np.minimum` استفاده کنید).

4. شمارشگر گام‌های روزانه (بردارسازی در برابر حلقه / Vectorization vs Loop)

```
daily_steps = np.array([4200, 8100, 12500, 3000, 9800, 15200, 6700])
```

یک تابع مبتنی بر حلقه (Loop) بنویسید که مجموع گام‌های طی‌شده را به‌صورت روز به روز (مجموع انباشته یا Cumulative Sum) برگرداند. سپس تابع معادل آن را در NumPy پیدا کنید که همین کار را تنها با یک دستور انجام می‌دهد. خروجی هر دو روش را مقایسه کنید تا از یکسان بودن آن‌ها مطمئن شوید و به اختصار توضیح دهید چرا روش برداری (Vectorized) ترجیح داده می‌شود.

5. تصحیح کوییز کلاسی (`np.where`)

```
quiz_scores = np.array([45, 78, 92, 60, 55, 88, 30])
```

بدون نوشتن حلقه یا دستور `if/else` و با استفاده از `np.where`، یک آرایه جدید از برچسب‌ها بسازید؛ به این صورت که برای نمرات ۵۰ و بالاتر برچسب "pass" (قبول) و برای بقیه "fail" (مردود) ثبت شود. در نهایت تعداد دانشجویانی که قبول شده‌اند را چاپ کنید.

6. ردیاب قیمت سهام (تغییرات درصدی + محاسبات آماری)

```
prices = np.array([100, 102, 98, 105, 110, 107, 115])
```

الف) درصد تغییرات قیمت روزانه را با استفاده از عملیات برداری (بدون حلقه) محاسبه کنید. برای این کار نیاز است که آرایه را طوری تغییر جهت یا شیفت دهید که دو روز متوالی مقابل هم قرار گیرند.

ب) میانگین و انحراف معیار این تغییرات روزانه را به دست آورید و چاپ کنید که در کدام روز، بیشترین جهش قیمتی رخ داده است.

🎁 امتیازی (برای گروه‌هایی که زمان اضافه دارند)

هر 7 شهر را بر اساس weighted distance نسبت به سفر ایده‌آل‌تان، از بهترین تا بدترین رتبه‌بندی کنید و یک leaderboard چاپ کنید.